

Bloom Filters – Empirical Study

James Pozzolungo - 14502850

GitHub Repository:

https://github.com/jamespozzolungo1/AdvancedAlgorithms_Assignment1

Section 1 – What I Built

A bloom filter is a space and time efficient probabilistic data structure that is used to represent a set of elements and perform set membership queries whilst allowing for a small probability of error (Geravand & Ahmadi, 2013). For this instance, the filter stores an array of m bits and adding an item hashes it to k bit positions and sets those bits to 1. Checking an item looks at the same k positions, if any position's bit is 0, it can then be concluded that the item was never added; if all position's bits are 1, it can be concluded that it is probably added. This is due to potential overlap of over item's big positions ultimately meaning that false positives are possible. The primary language for this project was C++, being used in all files excluding plotting (plot.py). Familiarity and frequent usage of this language for work on data structures and algorithms helped influence this choice, along with this, it made it possible to use `std::unordered_set` as something to compare the bloom filter against, while python was used for plotting strictly due to its simplicity. Data is represented through a Boolean type vector which packs 8 bits per byte. Instead of k separate hash functions, double hashing is used to generate all k positions from just two hashes, h_1 and h_2 (Kirsch & Mitzenmacher, 2006). Hashing through `FNV-1a + mix()` was chosen as a design decision instead of `std::hash` due to the built-in hash being compiler dependent, meaning the potential for inconsistent results. Strings being the only permitted datatype allowed for improved simplicity, along with deletion not being implemented since clearing bits would cause false negatives. Other minor simplifications include fixed size, and the h_2 guard only handling if $h_2 = 0$ to stop repeats. Bloomfilter.h consists of the algorithm's implementation, testbloom.cpp consists of 7 tests of the algorithm, with 3 edge-case oriented, experiments.cpp consists of 4 experiments, with 1 being on the baseline comparison of `std::unordered_set`, and plot.py being used to plot the CSV's generated in experiments.cpp.

Section 2 – Implementation and Empirical Study

The measured false-positive rate of the bloom filter was compared against the theoretical rate $p = (1 - e^{(-kn/m)})^k$ (Broder & Mitzenmacher, 2004). The filter was also compared against `std::unordered_set<std::string>` as a baseline, with specific focus on both memory use and speed.

Hypotheses Include:

- i. The measured false-positive rate will closely match theory as the number of items n increases.
- ii. The false-positive rate against k will form a U-shape, with its lowest point near the theoretical best $k = (m/n) * \ln 2 \approx 6.93$ (Broder & Mitzenmacher, 2004).
- iii. A prime m will match theory more closely than a composite m , since double hashing can repeat positions when $h2$ shares a factor with m .
- iv. The bloom filter will use far less memory than `std::unordered_set` and have faster lookups.

For each data point, n items (*item0*, *item1*, ...) were inserted into a new filter. 20,000 different items that were never inserted were checked, and any “probably yes” was counted as a false positive.

- Experiment 1 used $m = 10,000$, $k = 7$, with n from 100 to 2,000
- Experiment 2 used $m = 10,000$, $n = 1,000$ with k from 1 to 15
- Experiment 3 used $m = 10,000$, and $m = 10,007$ (prime), $n = 1,000$, k from 1 to 15, averaged over 5 trials, with different item names with each trial
- Experiment 4 sized the filter for a 1% false positive rate, with n from 1,000 to 1,000,000. For memory, every heap allocation made by `std::unordered_set` was counted by replacing operator `new`, while bloom filter’s memory was calculated as its object size + $m/8$ bytes. Timing was conducted through `std::chrono`.

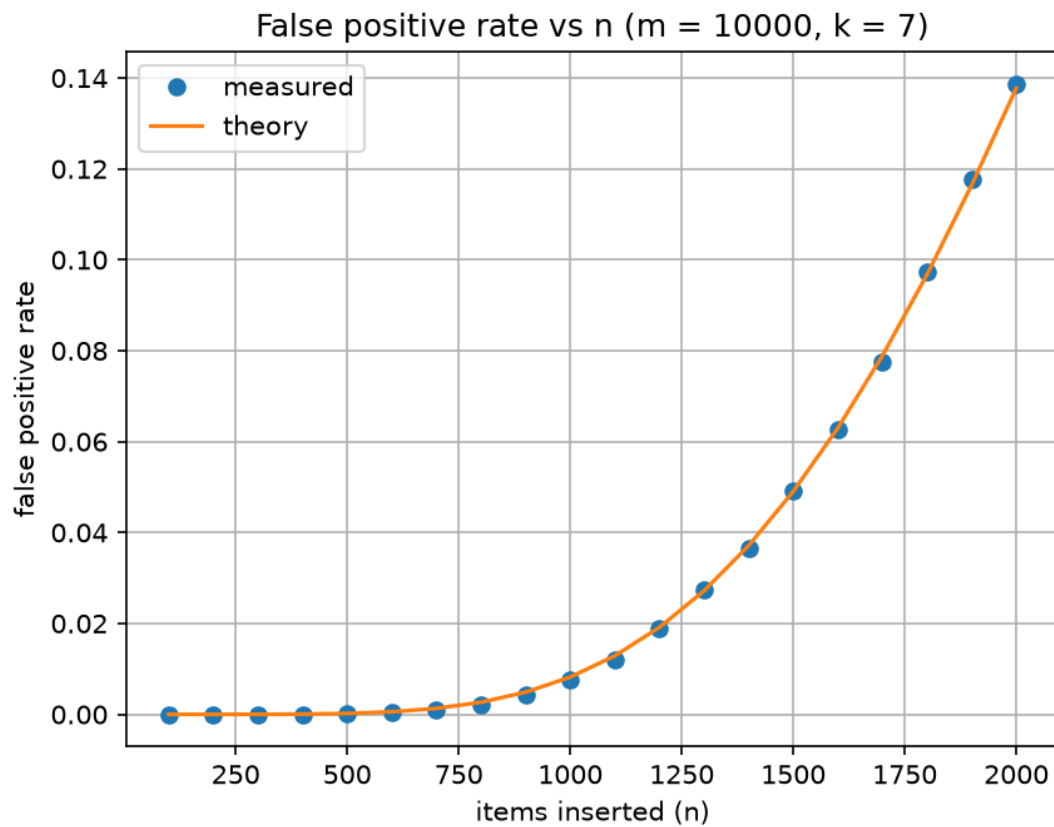


Figure 1: False-positive rate vs n ($m = 10,000$, $k = 7$)

The first experiment overall supports the first hypothesis, as the measured rate matched theory closely across all data points. For $n < 400$, the measured rate was 0, as through 20,000 tests, rates below about 1 in 20,000 can't be detected.

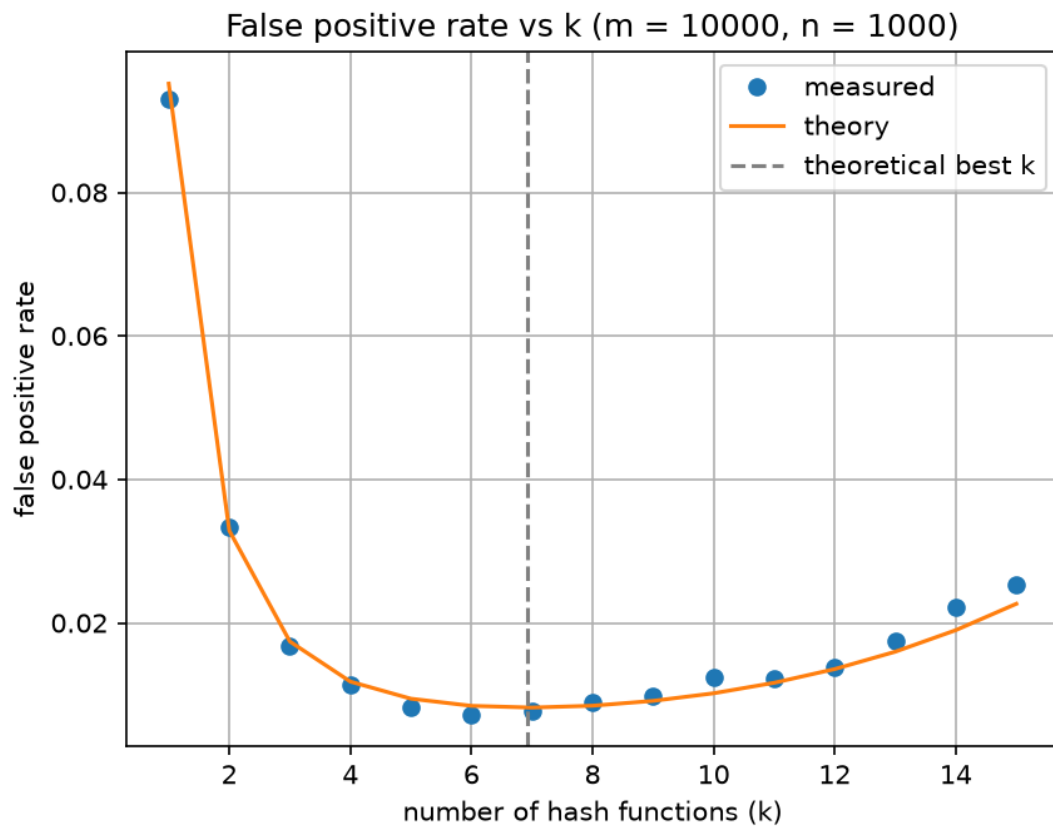


Figure 2: False positive rate vs k ($m = 10,000$, $n = 1,000$)

The second experiment also supports the second hypothesis, showing the same expected U shape. The lowest measured rate was at $k = 6$, with the theoretical best being 6.93. While at a higher k the measured rate sat above theory. As this experiment was conducted through a single run, some variation has the potential to be a result of noise.

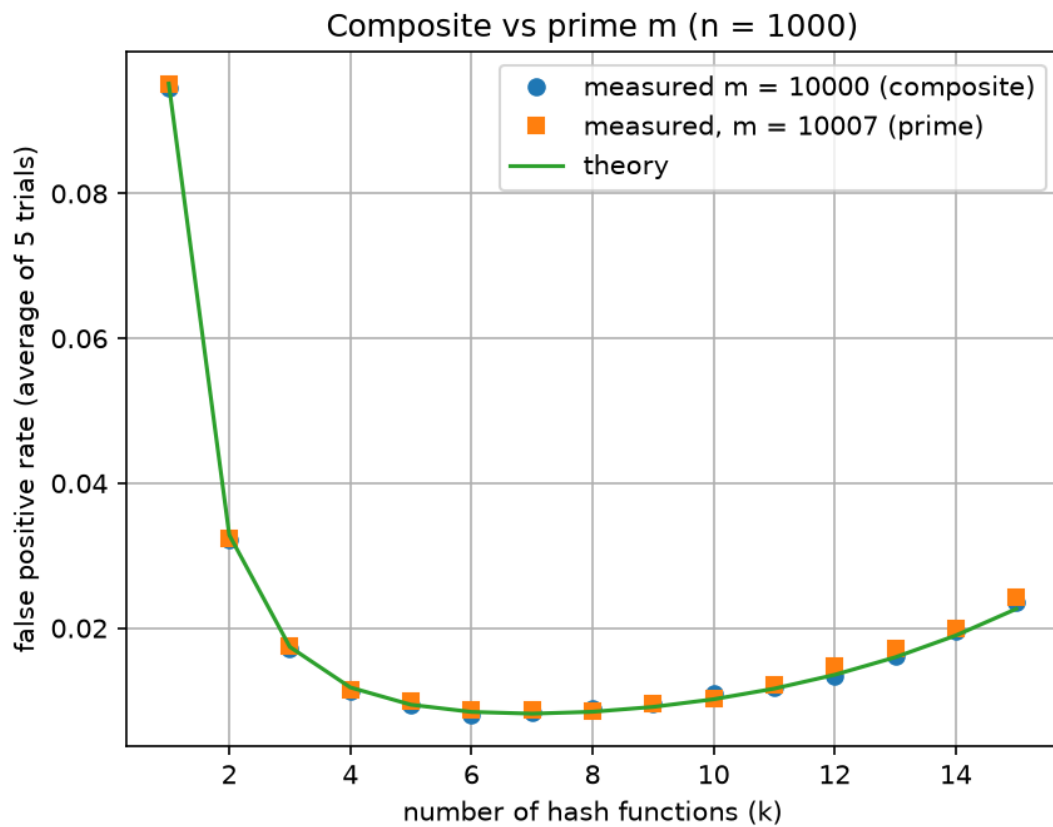


Figure 3: Composite vs prime m, averaged over 5 trials (n = 1,000)

The following experiment doesn't seem to be as supportive of the hypothesis, as the prime m wasn't consistently seen to be closer. At $k \approx 14$, both sat about 3-7% above theory. Since this repeats experiment 2's setup over 5 trials, the gap at $k = 14$ shrinks from about 16% in experiment 2 to about 3%, suggesting figure 2's deviation was most likely a result of noise.

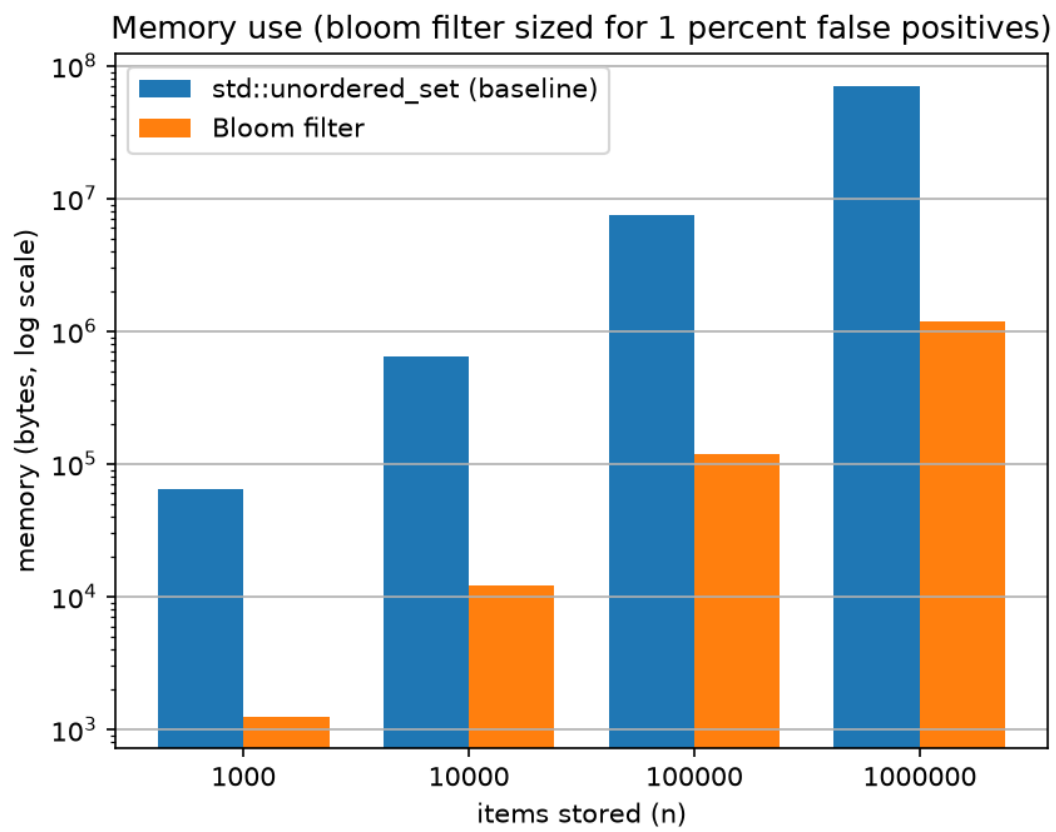


Figure 4: Memory use, bloom filter vs `std::unordered_set` (log scale)

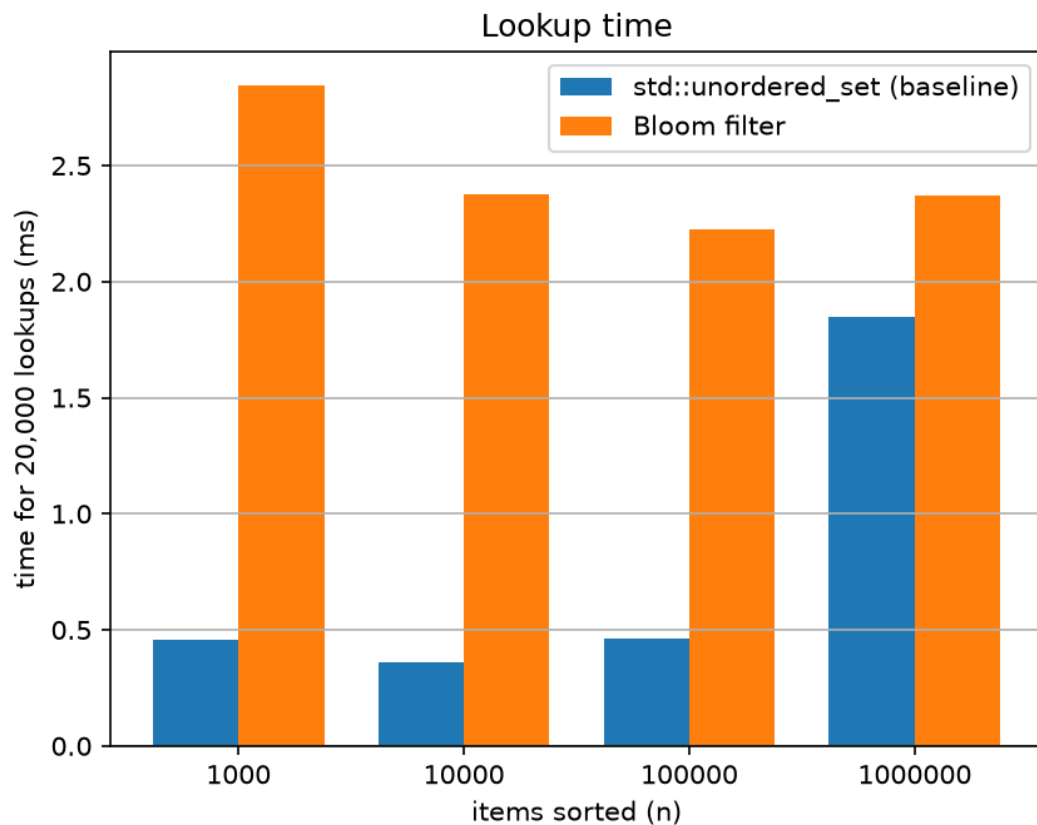


Figure 5: Time for 20,000 lookups, Bloom Filter vs `std::unordered_set`

The fourth experiment seems to partly support the fourth hypothesis. The bloom filter seems to use 52-63x less memory than the unordered set, with its false positive rate staying between ranges 0.82% and 1.05%. However, the unordered_set had faster lookups at every size, including $n = 1,000,000$ (1.85ms compared to 2.37ms). The gap grew as n shrank, with unordered_set being 6x faster at $n = 1,000$. However, for inserts specifically, the bloom filter was about 2x faster at $n = 1,000,000$ (114ms compared to 229ms).

The similar scores in experiments 1 and 2 suggest that the hashing behaves close to the random independent positions the formula assumes. The measured rate sitting above theory at high k could be due to the standard formula being an approximation rather than a pinpoint rate (Bose et al., 2008). Experiment 3 shows it isn't caused by repeated positions from a composite m , since there was no measured consistent difference.

The speed results show the bloom filters shines in aspects regarding memory opposed to speed. A possible reason for this being positions() creating a new vector every time it's called, thus adding a heap allocation to every lookup, whilst `std::unordered_set` only computes on hash and checks one bucket, whilst the bloom filter has to compute 7 positions and check 7 bits per lookup. As n grows, the unordered set slows down as memory usage grows, potentially explaining the discrepancy. As mentioned previously, a fixed hash function allows

for reproducible results on other machines/compilers, instead of built in hash's such as `std::hash`. It is also possible that there may be small noise across experiments 1, 2 and 4 due to a single run per data point approach. Slight variation in timing results may also occur between machines resulting in difficulty for exact replicability, meaning speed results should be treated more as as rough comparisons.

Section 3a – What I learned

I was relatively surprised when I saw that the unordered set actually ran faster while using more memory the lower n is, with the biggest gap in speed being when $n = 1,000$, with only a 1.3x difference when $n = 1,000,000$. It's something I haven't really considered before and always assume less memory = faster as a general rule of thumb. An important lesson I had learnt that I wouldn't normally consider is that passing tests doesn't always mean the code is correct. This was evident in this project when I had accidentally written "=" instead of "+" in `positions()` and `mix()`, resulting in the compiled code to pass every test whilst having a false positive rate of 100%. My tests I had at that moment only checked that there were no false negatives which allowed the bug to bypass. The implementation of `testFalsePositiveRate` helped catch this and I ultimately had learnt the importance of understanding what tests need to check what the code is actual for.

Section 3b – AI use

Claude was moderately used for both the conceptual understanding of bloom filters along with the implementation work. It was used for more specifically assistance in scaffolding the bloom filter, tests/experiments, rough layout of the report, generating matplotlib plotting code, debugging and explaining/teaching the conceptual side of bloom filters (e.g. how it works, double hashing, false-positive formula) through standard discussion and quizzing. The initial suggested implementation of C++ hash by Claude was unideal, in which a raw FNV-1a was suggested, which clumped similar keys like *item1* and *item_2* together and pushed the $k = 1$ false positive rate about 15% below theory, a solution to this was the implementation of the `mix()` function in on top of the base hash. Along with this, through the tests with Claude, no tests checked if the filter actually filters; this resulted in a bug from the typo mentioned in Section 3a resulting in a bug with 100% false positive rate on every test while passing, which was then fixed through the addition of `testFalsePositiveRate`. Due to the simplicity of the chosen algorithm, I feel comfortable with majority of the content, however, relying on AI for this assignment has made me a bit less confident in some parts of the work than if I had been more hands on in the development. 2 specific examples would be the hash constants in `hashString()` and `mix()`; and the operator new memory-counting being taken largely on trust.

Section 4 – Video Walkthrough

<https://youtu.be/FtIIDkTAyPs>

Reference List:

- Bose, P., Guo, H., Kranakis, E., Maheshwari, A., Morin, P., Morrison, J., Smid, M., & Tang, Y. (2008). On the false-positive rate of Bloom filters. *Information Processing Letters*, 108(4), 210–213. <https://doi.org/10.1016/j.ipl.2008.05.018>
- Broder, A., & Mitzenmacher, M. (2004). Network Applications of Bloom Filters: A Survey. *Internet Mathematics*, 1(4), 485–509. <https://doi.org/10.1080/15427951.2004.10129096>
- Geravand, S., & Ahmadi, M. (2013). Bloom filter applications in network security: A state-of-the-art survey. *Computer Networks*, 57(18), 4047–4064. <https://doi.org/10.1016/j.comnet.2013.09.003>
- Kirsch, A., & Mitzenmacher, M. (2006). Less Hashing, Same Performance: Building a Better Bloom Filter. *Lecture Notes in Computer Science*, 456–467. https://doi.org/10.1007/11841036_42